CaliberGraph: Bounded Verification of Scalar Aggregates through Join-Multiplicity Encodings

Hongsheng Wang, Dazhen Deng

Abstract

For bounded verification of scalar SUM/COUNT over separable inner-star joins, we implement known aggregate factorization with exact leaf-match counts and compare products with finite-domain selectors. The fragment admits root-only integer arithmetic and BETWEEN with explicit NULL semantics. Assignment-wise derivations cover the abstract transformation; a common adapter separately audits a narrow source-contract AST, while query parsing remains a trust boundary. Across 28 exposed configurations, 168 six-arm cost slots yield 127 strictly accepted pair decisions, and 168 individual-query checks yield 115 strict bounded agreements. Unknowns, contract refusals, replay-domain uncertainty and monitoring failures retain their denominators. Flat and selector costs are close, and simple conjunction normalization also resolves a case missed by two official controls; no selector-specific advantage is established. A complete cached-AST scan admits only six of 48,910 public query slots, all without joins. An executed constraint-direction counterexample motivates the adapter but does not measure natural defect prevalence. The evidence establishes a restricted implementation and its unresolved boundaries, not a new factorization identity, general correctness proof or broadly useful optimization.

Keywords: SQL equivalence, bounded verification, bag semantics, scalar aggregation, SMT encoding

Internal integration draft, 11 October 2026. The ten-page working target is an author planning choice, not a verified journal limit. The final v5 evidence cut is integrated; compilation and independent page review are recorded separately. Scientific readiness remains **NO-GO**: novelty and a useful comparative benefit are not established. No submission is authorized.

1. Introduction

Nullable arguments distinguish `COUNT(x)` from `COUNT(*)`; duplicate joins multiply measures; empty scalar SUM returns NULL. Testing one database can miss these differences. Bounded equivalence verification instead searches a declared tuple domain under supplied constraints, as implemented for complex SQL by VeriEQL (He et al., 2024).

For a root tuple joined independently to several leaves, exact leaf-match counts multiply its contribution. We implement this known factorization as a product and compare a selector that substitutes guarded constant multiples with shared variables. Selectors remove one source of variable multiplication but add branches; measures and ICs can remain nonlinear. Benefits must survive known-factorization and preprocessing controls, including frontend, fallback and witness-checking costs.

Our object is fixed-query verification over a constructed INT-schema star fragment, not end-to-end NL2Metric or business-intent certification. Earlier CaliberGraph metadata scores do not validate it. We provide assignment-wise derivations, code using official base variables, a narrow contract-AST adapter and failure-preserving evidence. None certifies the text parser or the complete implementation. Public eligible pairs contain no join, so applicability and a useful new contribution remain open. Sections 2–4 define prior work and the method; Sections 5–7 report implementation and evidence.

2. Related work and contribution boundary

SQL equivalence and counterexamples. VeriEQL encodes bounded SQL and integrity constraints, returning a counterexample or a verdict within that domain (He et al., 2024). Its official path is our baseline; a local Cartesian emitter

is only a diagnostic control. Cosette already combines symbolic tuple multiplicities, counterexamples and proof-oriented reasoning (Chu et al., 2017). SQLSolver reasons about unbounded summations using an extension of linear integer arithmetic (Ding et al., 2023); our finite tuple domain offers a different, narrower guarantee, not demonstrated superiority.

Factorized aggregates. Factorized representations exploit distributivity and sharing (Olteanu and Schleich, 2016). Bakibayev et al. (2013) explicitly describe count products and partial-count/partial-sum combinations; FAQ also folds common factors (Abo Khamis et al., 2023). Neither factorization nor finite-domain case splitting is new. The candidate contribution is their integration and measured tradeoff under a common bounded contract, requiring known-product and preprocessing controls.

Bounded under-approximation search. Polygon uses conflict-driven under-approximation search (Zhao et al., 2025). A completed full search returning no input for unequal outputs establishes bounded equivalence over its at-most-row domain; timeout or one UNSAT under-approximation does not. Its search/precise modes support ICs. Our exact positive cardinalities differ, but this is not novelty; a matched performance comparison remains open.

Measurement semantics. Thanisch et al. analyze measurement issues in SQL arithmetic and aggregation (Thanisch et al., 2019). The available abstract describes numerical properties including units, scales and grouping, making it a relevant DKE semantic-error precedent. We instead study bounded relational-result equivalence under supplied constraints. These objects may be complementary. Having inspected the abstract rather than the full paper, we cannot claim absent NULL rules, relational analyses or optimizations, or describe its checks as merely syntactic.

Text-to-SQL verification. SpotIt+ searches for distinguishing databases and mines constraints for text-to-SQL evaluation (Tremante et al., 2026). Our fixed-constraint runs do not reproduce its complete pipeline or model calls. The historical direct-engine adapter's automatic route uses `SetSemanticsVerifier`. One-row scalar equality coincides with bag equality when NULL and row preservation agree; this does not make that route a bag implementation. Unverified comparison details remain unknown rather than absent capabilities.

3. Task, domain, and supported SQL

3.1. The fixed verification contract

An input is $\mathcal{I} = (Q_1, Q_2, \mathcal{S}, C, \mathbf{N}, \delta)$: two original SQL strings, the complete schema including type spellings, the complete registered integrity-constraint AST, an exact table-size vector, and the dialect/configuration. Let $\mathcal{D}(\mathcal{I})$ denote the databases represented by the verifier's emitted base facts. This encoded domain need not equal the intended source-constraint domain; Section 7.6 gives an executed v4 counterexample. V5 adds the bounded AST bridge in Section 5.2, with a narrower explicit IC grammar. It does not certify the preceding text parser. The current Environment asserts every created tuple live, so N_T means exactly N_T rows, not at most N_T. One vector does not establish smaller or unequal vectors.

The verification goal is

$$\exists D \in \mathcal{D}(\mathcal{I}) : \quad Q_1(D) \neq_{\text{bag}} Q_2(D). \tag{1}$$

A satisfiable result supplies a candidate distinguishing database. An unsatisfiable result excludes a difference only in this declared domain, subject to faithful encoding. We first require the complete base contract to be satisfiable, so an inconsistent specification is not reported as useful evidence of equivalence. Failure to establish that guard is retained as a distinct outcome.

Type strings remain unchanged. The executed harness requires every physical schema column to be `INT`, whose inspected official encoding bounds inputs to signed 32-bit integers. Other integer spellings are recognized in some code paths but are not interchangeable experimental contracts. Aggregate expressions use mathematical integers. Even INT32 inputs can overflow a concrete expression or aggregate; Section 5.4 gives the separate SQLite replay guard. Full mixed-type schemas and general DBMS overflow semantics are not supported by these experiments.

3.2. Recognized fragment

Runtime-v4 accepts one scalar output, `SUM(e)`, `COUNT(e)`, or `COUNT(*)`. The root-only expression e is built from integer literals, root columns, binary $+, -, *$, and unary signs. FROM has a first root relation and zero or more distinct physical leaves; every leaf needs a column equality to the root. ON/WHERE is a conjunction of integer comparisons, NULL tests, and BETWEEN, with each atom reading at most the root and one leaf. Predicate operands are columns or integer literals; arithmetic predicate operands and NULL literals are not admitted. Runtime-v2/v3 accepted only a root-column aggregate argument, so their recorded public coverage cannot be transferred to v4.

Aliases must be unique after case folding; repeated physical relations, including self-joins under different aliases, are rejected. Under the official double-underscore encoding, we enumerate schema-backed interpretations of both `alias__column` and unqualified columns and require exactly one match in their union. We also reject unsupported types/expressions, multiple outputs, GROUP BY, DISTINCT, outer joins, windows, correlated subqueries, cross-leaf predicates and OR. A no-leaf query has empty product one; both eligible public pairs have this form. Empty joined results are supported, but v5 rejects nonpositive base-row counts before any arm rather than using fallback.

SQL predicate truth is three-valued. Only TRUE admits a tuple. BETWEEN(a, b, c) is TRUE exactly when all three operands are non-NULL and $b \le a \le c$; reversed bounds admit no row. For arithmetic, the result NULL flag is the disjunction of operand flags and the payload uses the corresponding mathematical integer operation. Structural induction gives strict NULL propagation, including $0 \cdot \text{NULL} = \text{NULL}$; a non-NULL zero still contributes to COUNT. No key, uniqueness, join truth, or non-NULL fact is inferred from names or data.

3.3. Scalar result representation

An encoded scalar aggregate is (z, v, A), where z is its NULL flag, v is its integer payload, and A contains fresh-variable definitions. The numeric payload of a NULL result is semantically irrelevant. Two one-row scalar outputs differ exactly when

$$(z_1 \neq z_2) \ \vee \ (\neg z_1 \wedge \neg z_2 \wedge v_1 \neq v_2). \tag{2}$$

The result row is preserved even when the aggregate input is empty. COUNT is never NULL; SUM is NULL when there is no non-NULL contributing tuple. Equating NULL with zero, or dropping that scalar row, changes the verification target.

4. Three join-multiplicity encodings

4.1. Flat reference semantics

Fix a database assignment D, root row f, and leaf row $d \in D_j$. Let $R(f)$ indicate root liveness and TRUE evaluation of all root-only predicates. Let $I_j(f, d)$ indicate leaf liveness and TRUE evaluation of every predicate assigned to leaf j. Define

$$C_j(f) = \sum_{d \in D_j} \mathbf{1}[I_j(f, d)], \qquad 0 \le C_j(f) \le N_j. \tag{3}$$

These are exact expressions over the same database variables, not free symbolic counts or estimates. Repeated rows occupy different tuple positions and contribute separately.

Let the root expression evaluate to $(z_e(f), v_e(f))$ and set $u(f) = \mathbf{1}[R(f) \wedge \neg z_e(f)]$. For COUNT(*) or a non-NULL literal use $u(f) = \mathbf{1}[R(f)]$. A local flat emitter enumerates every $(f, d_1, \ldots, d_k)$ and guards its contribution with all predicates. It shares the candidate recognizer, so it is not a fully independent semantic oracle.

4.2. Exact factorization

For every fixed root tuple, the separability condition makes the admitted leaf combinations a Cartesian product. Define $M(f) = \prod_{j=1}^{k} C_j(f)$, with $M(f) = 1$ when $k = 0$. The non-NULL contributing count is

$$n = \sum_{f \in F} u(f) M(f). \tag{4}$$

COUNT returns (false, n). For SUM the numeric payload and NULL flag are

$$v = \sum_{f \in F} u(f) v_e(f) M(f), \qquad z = (n = 0), \tag{5}$$

where guarded expressions ensure that a NULL payload is not treated as a real value.

Lemma 1 (Assignment-wise bag multiplicity)**.** *For a query in the stated fragment and any fixed database assignment, the flat join enumeration and Equations* (4)–(5) *have identical scalar NULL flags and integer payloads.*

Proof. Fix f. For each leaf j, form the list of tuple positions satisfying $I_j(f, d)$. A joined tuple with root f selects exactly one position from each list, and every such selection satisfies the conjunction because no atom refers to two leaves. There are therefore $\prod_j C_j(f)$ selections, including duplicate-valued positions. Each has the same root aggregate argument. Multiplying its non-NULL indicator and value by that count equals summing the corresponding flat contributions. Summing over root positions yields the equations. The contributing count is zero exactly when no non-NULL argument survives, establishing SUM's NULL flag and COUNT's non-NULL count.

This proof needs no probabilistic independence. Integrity constraints may correlate different leaves; for each database satisfying them, the query's fixed-root predicate still separates. Constraints remain in the solver and restrict which assignments are admitted. A query predicate such as $D_1.a = D_2.b$ destroys this Cartesian argument even if the schema is called a star, so the recognizer rejects it.

4.3. Products and bounded selectors

The product emitter introduces fresh variables for successive accumulators. For a current value a and an exact match count $C \in [0, N]$, it adds $a' = aC$. For SUM it propagates both a value accumulator and a contribution-count accumulator; COUNT needs only the latter. Definitions reference previous fresh variables rather than repeatedly inlining their expression trees.

The selector emitter instead binds

$$a' = \sum_{c=0}^{N} \mathrm{If}(C = c, c \cdot a, 0). \tag{6}$$

The count definition in Equation (3) supplies the finite range. An additional unconstrained count variable with a range assertion would not preserve the original query unless it were also bound to that exact sum.

Lemma 2 (Selector identity and conservative extension)**.** *If C is the exact integer count in Equation* (3)*, Equation* (6) *equals aC. With globally fresh auxiliary variables and every defining equality asserted, introducing the count and accumulator definitions neither removes nor adds assignments of the original database variables.*

Proof. Exactly one integer $c \in \{0, \ldots, N\}$ equals C. Its branch contributes Ca and every other branch contributes zero. For a fixed original assignment, evaluate count definitions and then accumulator definitions in order. This gives a unique value to each auxiliary variable. Conversely, the equalities force those values in every satisfying extension. Projecting away auxiliaries therefore recovers precisely the original assignment domain.

Induction across leaves, followed by Lemma 1, establishes equality of the abstract selector and flat scalar results. The argument includes negative values, duplicate matches, empty matching sets, and all-NULL aggregate arguments. It does not establish that the actual parser, name mapping, or emitter implements the premises correctly. Those are separate obligations in Section 5.3.

The selector removes variable products introduced by match multiplicities. Variable products inside $e(f)$ remain nonlinear, as may the unchanged constraints and official premises. Neither the full formula nor even the aggregate is automatically linear. No solver-complexity or termination improvement follows from the identity.

4.4. Local construction cost and a worked example

For N_F root rows and leaf sizes $N_1, \ldots, N_k$, local flat construction enumerates $N_F \prod_j N_j$ joined positions. Both factored forms construct $N_F \sum_j N_j$ leaf-match indicator terms. The selector adds $N_F \sum_j (N_j + 1)$ guarded branches for one accumulator and twice that many for SUM's two accumulators. Predicate size, complete constraint cost, parser work, and all other verifier costs are additional. The bound assumes explicit shared definitions; recursively substituting the previous accumulator into every branch invalidates the intended sharing argument.

A root value 3 with two matches in one leaf and three in another contributes 18: flat emits six threes, product computes $3 \cdot 2 \cdot 3$, and selectors choose the 2 and 3 branches. With no match in one leaf, the contribution count is zero and SUM is NULL, regardless of its payload. This is an analytical example, not a workload. At the executed two-root/two-leaf/two-row size, SUM has eight flat positions, eight factored match terms and 24 selector branches: there is no measured term-count reduction at that size, and asymptotics alone imply no speedup.

5. Implementation and failure contract

5.1. Frontend and complete fallback

The implementation uses VeriEQL v1.0, commit `3b99928b`, its original fast Z3 Python bindings, and a locally verified Z3 4.12.0.0 library. Runtime-v4 extends the candidate expression evaluator; v5 adds the common contract adapter below while preserving that kernel and the official source. The recognizer, three emitters, and scalar comparison reuse the official base variables and asserted facts. Fact hashes establish byte/formula identity, not faithful translation of raw integrity constraints or business meaning.

Figure 1 gives the routing boundary. Before parsing, the raw guard rejects any double quote, backtick or dollar sign anywhere in SQL, including literals and comments; this conservative refusal has no fallback. V5's physical table/column names must be uppercase ASCII identifiers without double underscores, with unique table–column encoding and first-separator roundtrip. Unsupported or unverified ICs also stop both routes. After admission, candidate ineligibility uses the complete official query path on the same schema, rows and audited contract. Solver unknown falls back only with at least 500 ms remaining; otherwise it stays unknown. Exceptions and invalid witnesses remain explicit.

The cost harness constructs its common Environment with the MYSQL dialect. Public applicability instead uses previously stored ALL-dialect ASTs. These are separate configurations, not evidence of cross-dialect equivalence. The earlier unequal-budget runtime-v2 ran official first; later independent-process versions retain separate result identities.

5.2. A common source-AST contract bridge

The direction failure in Section 7.6 motivates a common adapter before any comparison arm. Its accepted IC AST is absent or a conjunction list, allowing nested AND, non-NULL tests, one column compared with one integer literal in either order using six comparison operators, and inclusive column BETWEEN integer bounds. Every clause must be TRUE on every physical row of its referenced table; NULL fails. This explicit contract is narrower than the upstream interface and does not infer SQL CHECK semantics. One unsupported node refuses the whole contract without fallback.

Let B be the fresh Environment's schema, exact-row facts and bounds before adding the tested IC. A separate reference emits C_r from the original AST on the same base symbols. A normalized copy reverses literal-left ordered comparisons and expands BETWEEN into inclusive column-left bounds; only this copy enters the actual upstream compiler, whose added facts form C_u. Original AST and per-leaf transformations are retained. Admission requires

$$\mathrm{SAT}(B), \qquad \mathrm{UNSAT}(B \wedge (C_u \oplus C_r)). \tag{7}$$

Neither tested interpretation is asserted in the XOR premises, where it could hide mapping counterexamples. For the XOR audit, SAT or unknown refuses admission. Base SAT prevents a vacuous audit; a separate later guard requires $B \wedge C_u$ satisfiable and excludes deleted rows. Contradictory ICs are not query-equivalence successes.

Every arm audits the same registered IC in a fresh Environment. Fallback and differentials require its attached bridge record and unchanged complete-fact digest. These local identity checks are not portable certificates, and

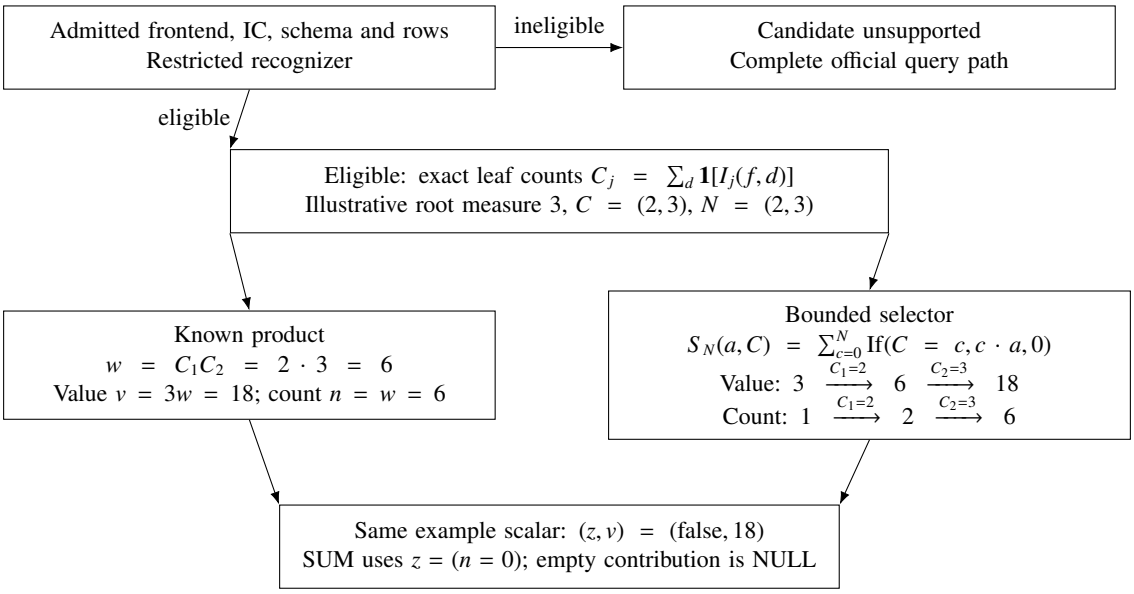


Figure 1: Alternative encodings for one illustrative, non-NULL root value with $R(f) = \text{true}$. Exact C_j counts actual leaf positions with TRUE predicates, not free variables. The leaf sizes/count bounds $N = (2,3)$ are illustrative, not the executed two-leaf/two-row setup. Product and selector produce the same SUM value and non-NULL contribution count; local flat enumerates six joined positions. NULL arguments contribute nothing, and zero contributions yield NULL SUM. Unsafe frontend normalization or an unverified IC stops before both routes; only candidate ineligibility uses the official fallback. The example supplies no measured cost or speed claim.

the official arm uses an explicitly controlled IC adapter. A concrete evaluator checks every SAT witness against the original AST before the integer-domain audit and unchanged SQL replay. All checks count toward the process budget.

Equation 7 establishes bounded equality of the two interpretations, conditional on the base representation and reference implementation. The reference shares base variables and solver primitives but not the upstream IC compiler. This does not certify text parsing, arbitrary ICs, business truth or query translation. Executed audits are input-specific; old v4 runs inherit no bridge guarantee.

5.3. Mapping obligations

Constructed column pairs `V$X`/`V_DOLLAR_X` and `A.B`/`A____B` produced identical ASTs/plans although original SQL returned 2 and 7. No solver ran in these diagnostics: they expose lossy parsing, not an observed solver verdict. Ten later guarded slots rejected both cases before fallback; 35 unselected routing slots remain not run. An earlier six-slot name-mapping rejection is also retained.

Table 1 maps definitions to code. Differentials check consistency, exact-row liveness, scalar shape, fresh definitions and NULL/value difference. Their common query encoding remains a trust boundary despite the separately audited IC AST.

Object	Implementation	Boundary
Fragment, e	`recognize`, `value`	Root arithmetic, strict NULL; complete INT schema
R, I_j	`true_predicate`	TRUE-only atoms; no cross-leaf predicate
C_j, M	`encode`	Exact row indicators and match-count products
Fresh definitions	Query/mode-specific prefix	Acyclic total definitions; differential checks
Scalar difference	`unequal`	NULL payload ignored; result row preserved
Contract AST	Common bridge + reference	Base SAT; contract XOR UNSAT; source AST only
Concrete witness	Original IC + domain + SQL	Original IC, integer domain, then raw SQL; common-plan oracle boundary

Table 1: Definition-to-code map. Local checks do not promote the abstract lemmas into a theorem of the complete SQL implementation.

5.4. Concrete replay domain

SMT uses mathematical integers; SQLite may promote overflowing expressions to REAL or reject overflowing SUM. Before replay, the common auditor checks base values, expression intermediates and weighted aggregates. For flattened associative expressions it bounds positive/negative addition totals and absolute nonzero products across parenthesizations; SUM is checked across row orders. Unsafe or unproved cases remain `UNKNOWN_ORACLE_DOMAIN`, without clipping, rounding or changed constraints. This conservative check can reject a safe concrete order. Only admitted witnesses replay the complete original SQL, preserving duplicates and NULL. The auditor shares the candidate plan and is not an independent semantic oracle.

All raw outcomes and separately derived acceptance categories retain their denominators. Contradictory contracts stop before query comparison; concrete replays and finite differentials do not prove parser totality or complete compiler correctness.

6. Evaluation design

All cases are exposed development material, without held-out data or human gold. Six original cases cover conjunction order, nullable COUNT, non-NULL ICs, SUM/COUNT differences, empty SUM and all-NULL SUM. At two row bounds they give twelve configurations; ten v4 expression controls and six v5 IC-adapter controls yield 28 configurations but only 22 logical cases. Six cost arms register 168 slots; two queries times three encoders separately register 168 differential slots. These repetitions are not independent samples. V4 and v5 remain separate contract versions.

The six arms are official, official with generic SMT preprocessing, local flat, known product, selector, and official with cheap AND normalization. The last recursively sorts admitted conjunction operands: identical normalized ASTs give a conditional result; otherwise official fallback runs. It is not a full optimizer and shares the frontend boundary. Each arm uses a fresh process/Environment, common native library and complete input, CPU soft/hard limits of 14/15 seconds, a 25-second supervised wall limit and 1 GiB RSS limit. Startup, IC audits, all query work, fallback, replay and cleanup count toward cost. Solver threads are one; auxiliary monitoring threads exist. One pass does not establish statistical speedup.

Applicability covers 397 Calcite, 64 literature and 23,994 LeetCode pairs, retaining all 48,910 query slots and duplicates. The v4 recognizer consumes saved official ALL-dialect ASTs and errors. It does not reparse SQL, reconstruct missing ASTs, assert ICs, encode, solve or execute queries. This is a cached recognizer scan, not parser validation or equivalence accuracy. Original unequal-budget runs and fixed-point checks remain separate diagnostics.

7. Results and retained failures

7.1. Complete v5 outcomes

Table 2 accounts for every selected slot. A strict query result requires the recorded bridge and full-contract gates, complete worker/cleanup receipts, acceptable process supervision and, for SAT, original-AST validation and safe original-SQL replay. Typed refusals remain separate from query decisions. These counts describe exposed development configurations, not independent benchmark accuracy.

Run	Slots	Strict	Unknown	Domain	Typed	Monitor
Six-arm cost	168	127	7	6	24	4
Query differential	168	115	23	0	24	6
SpotIt engine	28	21	2	1	3	1

Table 2: Mutually exclusive v5 categories. Strict means a valid pair decision for cost/SpotIt, or bounded encoder agreement for differential. Domain denotes an unproved replay domain; Typed is a verified expected no-query refusal; Monitor is unavailable RSS observation, not established memory exhaustion. No failed category is an equivalence success.

The cost records contain 78 raw UNSAT, 52 SAT, eight unknown, six oracle-domain unknowns, eighteen refusals for inconsistent contracts and six for unsupported contracts. Four monitoring stops override raw outcomes: two SAT,

one UNSAT and one unknown. Thus strict decisions are 77 UNSAT and 50 SAT, not all 130 raw decisions. All 52 stored SAT witnesses pass original-contract and SQL replay checks; the two monitored attempts remain excluded from strict results. Across the prior twelve configurations, ten arithmetic controls and six adapter controls, strict pair counts are respectively 62/72, 47/60 and 18/36; the last group also has eighteen valid typed refusals.

Recovery retains 169 cost attempts for 168 slots: terminal slot 51 comes from the old batch, resumed 52–53 from the new one. Interrupted 52 and the earlier not-run 53 remain recorded. Interrupted 52 lacks full cost; its 1.234-second CPU checkpoint is only a lower bound. Stale raw pair-work fields are preserved; separate start, final-phase, cleanup and reaped-process receipts derive execution state without turning unknown/monitored outcomes into success.

7.2. Contract controls and differentials

For each cost and differential campaign, 162 inputs record base-only SAT and contract-XOR UNSAT; the six unsupported-contract controls stop before either check. Eighteen admitted contracts are contradictory and stop before query work. These input-specific checks do not certify arbitrary contract translation. In D01, $5 < T.A$ together with $T.A \leq 6$ forces the integer value 6: all six arms produce a strict SAT witness, validate the original AST and replay NULL versus 6. D02 yields six UNSAT decisions for an identical query; D03 yields six SAT decisions without explicit ICs. D04/D05 reject twelve inconsistent contracts, and D06 refuses six whole contracts containing an unsupported disjunction. All are constructed repair controls, not naturally collected defects.

Individual-query records contain 120 raw UNSAT differentials, 24 unknown and 24 typed refusals. Five raw UNSAT and one unknown coincide with monitoring stops, leaving 115 strict agreements. Flat, product and selector have respectively 39, 37 and 39 strict agreements out of 56 registered slots each. No validated differential counterexample is recorded, but the unknowns and refusals prevent a complete mapping result. These checks remain relative to shared query encoding and cannot replace the abstract premises with a general compiler theorem.

7.3. Costs and comparison boundary

Table 3 reports complete process costs for the 168 selected returned slots, including failures and refusals. The official arm also has an interrupted attempt whose full cost is unknown and is not included in these totals. Selector and flat totals are close, their strict counts differ, and each arm contributes one selected returned observation per configuration. No speedup, stability or selector-specific advantage follows.

Arm	Strict/28	Unknown	Monitor	CPU sum (s)	Median (s)
Official + IC adapter	20	2	1	62.612	1.315
Official + generic SMT	21	2	0	63.180	1.354
Local flat	23	0	0	37.110	1.314
Known product	19	3	1	73.661	1.336
Local selector	22	0	1	36.793	1.333
Official + cheap AND	22	0	1	50.485	1.332

Table 3: V5 cold-process CPU, 28 observed returns per arm. Each arm additionally has one domain refusal and four valid typed refusals. Official has a 29th, interrupted attempt with unrecorded full cost; its sum is incomplete campaign cost. Medians are across different outcome mixtures, not repeated trials.

For S01 at N4, official and generic-SMT return unknown; flat, selector and cheap AND return strict UNSAT at 1.314, 1.312 and 1.298 seconds CPU. Product records UNSAT at 1.361 seconds but has a monitoring stop and is not strict. Standard conjunction normalization therefore explains a success that cannot be uniquely attributed to selectors.

The controlled SpotIt engine selects 28 slots from its 168-slot enclosing manifest; 140 other-arm slots are unselected, not failures. Its 21 strict results comprise twelve UNSAT and nine replayed SAT. One raw inconsistent-contract refusal has a monitoring stop, leaving three valid typed refusals. CPU sums to 63.881 seconds. All 28 source-backed dispatch records identify AUTO_SET with all-null deletion disabled. This uses reconstructed native Z3 4.12 despite outer requirements specifying 4.15 and does not reproduce the full SpotIt+ mining pipeline. Scalar one-row set/bag coincidence does not change the engine implementation. Polygon's search and precise modes remain unbenchmarked under a matched domain.

7.4. Historical v4 and fixed-point evidence

V4 separately records 72 UNSAT, 41 SAT, seven unknown and twelve refusals across 132 cost slots. Six contract refusals retain raw NOT_RUN/BLOCKED_OR_ERROR. E01–E08's 48 decisions match expectations; eighteen SAT witnesses replay. Their 48 raw UNSAT differentials have three RSS-observation anomalies, versus one among 36 earlier v3 checks. These inherit no v5 bridge or strict-acceptance guarantee.

On twelve v4 configurations, flat/selector CPU was already close at 16.862/16.725 seconds with no unknowns. Product, official, generic and cheap AND took 52.130, 42.439, 42.410 and 30.133 seconds with 2, 2, 2 and 1 unknowns. Earlier N2 totals of 7.444–7.750 seconds also establish no selector advantage. Twelve fixed-point controls yield eighteen safe replays and three domain unknowns among 21 queries, not equivalence proofs. Historical SpotIt's 22 slots retain eleven UNSAT, seven SAT, two unknown, two refusals and an RSS anomaly. Full versioned records remain in the evidence package; they are not pooled replicates.

7.5. Complete public applicability

Table 4 includes all 24,455 public pairs. The six eligible slots correspond to four exact SQL strings or five SQL/full-context combinations, all with zero leaves. The two both-eligible pairs are an identical Calcite SUM pair and a literature COUNT comparison over one relation. Eligible public JOIN coverage is therefore zero, and LeetCode has no eligible slot. Its 23,994 pairs span only 56 source problem labels and 24,050 distinct SQL strings, not 23,994 independent tasks.

Source	Slots	Eligible	Ineligible	Refused	Unknown	Errors
Calcite	794	4	548	188	38	16
Literature	128	2	125	0	0	1
LeetCode	47,988	0	46,758	1	0	1,229
Total	48,910	6	47,431	189	38	1,246

Table 4: Full cached official ALL-AST scan using the v4 recognizer retained by v5. Refusals are raw frontend rejections; errors preserve original parse/AST-record failures. No slot is unprocessed. The scan does not execute the v5 IC bridge, reparse SQL or run a solver.

No eligibility filter or deduplication reduces the denominator. Two LeetCode blocks stopped after 1,973 and 1,978 recorded slots; resumes processed only the remaining 27 and 22, preserving earlier hashes and resource stops. Thirty-eight Calcite slots become UNKNOWN because cached AST objects cannot be safely reconstructed. The v4 extension adds no eligible query over v3. At pair level, two are both eligible, 24,412 definitively ineligible and 41 unresolved. This is applicability, not public equivalence accuracy.

Earlier 922-slot scans admit seven queries under v2/MYSQL and v2/ALL, versus six under guarded v3; these configuration changes are not independent tests. Static inventories found no original-fragment member among TPC-H's 22, SSB's thirteen or DuckDB TPC-DS's 99 definitions; official TPC-DS SQL was unavailable. Three SSB definitions became structural candidates only under combined arithmetic/BETWEEN/domain changes, each with one leaf. These static checks are not solver runs and provide no executed public join case.

7.6. Constraint-direction counterexample

A constructed diagnostic evaluates six literal-left operators at four values, including NULL: eight of its 24 cells disagree with the emitted official constraint. In particular, $5 < T.A$ is emitted in the opposite direction. With one exact row, the official checker reports UNSAT for `SUM(T.A) WHERE T.A<5` versus unfiltered `SUM(T.A)`. Yet the original contract admits $T.A = 6$, for which unchanged SQL returns NULL versus 6 in SQLite. This is an observed false-equivalence result under the original contract. The cells include fixed-model and substitution controls, not 24 independent verification problems or a defect-frequency estimate. V5's narrow AST adapter and D01 address this constructed direction error; they do not retroactively certify v4, arbitrary ICs or business truth.

8. Limitations, lineage, and reproducibility

Guarantees and lineage. The lemmas cover the stated finite relational fragment, excluding general grouping, outer joins, cross-leaf filters and unbounded equivalence. The IC adapter excludes foreign keys and functional dependencies and does not infer business intent, authorization or temporal correctness. Exact-row and at-most-row domains differ. Post-hoc fragment changes require new versioned evidence; known factorization and simple preprocessing remain essential controls.

Historical governance prompting scored 156/159 against CaliberGraph's 147/159; a dbt-derived result tied at 122/122 only with longest-alias linking, versus 120/122 for another linker. These do not evaluate this module. The legacy 1,108 distinct SQL attempts contain 1,074 connection/query timeouts and 34 cached first scalars; without complete same-snapshot references, all 1,488 model records remain semantically unknown, not 34/1,108 accuracy. No new business gold or human annotation is inferred.

The author states that this contribution does not duplicate SIGMOD847. Its inspected formal materials concern typed-delta refresh, reuse and version-consistent switchover. We do not borrow its algorithms, proofs or experimental numbers; ordinary additive aggregation and typed certificates are shared background. This records the author statement and inspected distinction, not an exhaustive originality proof.

Evidence. The local package retains original inputs, raw reports and decoded databases, with hashes, interrupted attempts and complete denominators. A claim ledger links paper values to saved rows. No live production system, external model API or GPU was used for this manuscript task; local historical artifacts were read. Third-party source identities remain recorded; the package is not a public release.

AI assistants performed code work, mathematical drafting, local experiment execution and orchestration, evidence analysis and manuscript preparation. Separate AI review is not human peer review. Author verification, affiliations, contribution/funding/conflict statements require actual confirmation. The official author order is Hongsheng Wang, then Dazhen Deng; no correspondence details are invented.

9. Conclusion

CaliberGraph implements known factorization and bounded selectors for a restricted scalar language. Abstract identities, IC audits, differentials and replays cover distinct conditional guarantees. Complete failure accounting and public exclusions leave novelty, useful applicability and selector benefit unresolved; the current evidence does not establish submission readiness.

References

Abo Khamis, M., Ngo, H.Q., Rudra, A., 2023. FAQ: Questions asked frequently. arXiv:1504.04044v7. URL: https://arxiv.org/abs/1504.04044v7.

Bakibayev, N., Kočiský, T., Olteanu, D., Závodný, J., 2013. Aggregation and ordering in factorised databases. arXiv:1307.0441v1. URL: https://arxiv.org/abs/1307.0441v1.

Chu, S., Wang, C., Weitz, K., Cheung, A., 2017. Cosette: An automated prover for SQL, in: Conference on Innovative Data Systems Research (CIDR). URL: https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf.

Ding, H., Wang, Z., Yang, Y., Zhang, D., Xu, Z., Chen, H., Piskac, R., Li, J., 2023. Proving query equivalence using linear integer arithmetic. Proceedings of the ACM on Management of Data 1, 227:1–227:26. URL: https://www.cs.yale.edu/homes/piskac/papers/2024SIGMOD.pdf, doi:10.1145/3626768.

He, Y., Zhao, P., Wang, X., Wang, Y., 2024. VeriEQL: Bounded equivalence verification for complex SQL queries with integrity constraints. Proceedings of the ACM on Programming Languages 8, 132:1–132:64. URL: https://arxiv.org/html/2403.03193v2, doi:10.1145/3649849.

Olteanu, D., Schleich, M., 2016. Factorized databases. SIGMOD Record 45, 5–16. URL: https://sigmodrecord.org/publications/sigmodRecord/1606/pdfs/03_principles_Olteanu.pdf.

Thanisch, P., Niemi, T., Nummenmaa, J., Niinimäki, M., 2019. Detecting measurement issues in SQL arithmetic expressions and aggregations. Data & Knowledge Engineering 122, 116–129. doi:10.1016/j.datak.2019.06.001.

Tremante, A., He, Y., Klopfenstein, R., Wang, Y., Narodytska, N., Wu, H., 2026. SpotIt+: Verification-based text-to-SQL evaluation with database constraints. arXiv:2603.04334. URL: https://arxiv.org/abs/2603.04334, doi:10.48550/arXiv.2603.04334.

Zhao, P., Wang, Y., Wang, X., 2025. Polygon: Symbolic reasoning for SQL using conflict-driven under-approximation search. Proceedings of the ACM on Programming Languages 9, 1315–1340. URL: https://arxiv.org/html/2504.06542v1, doi:10.1145/3729303.